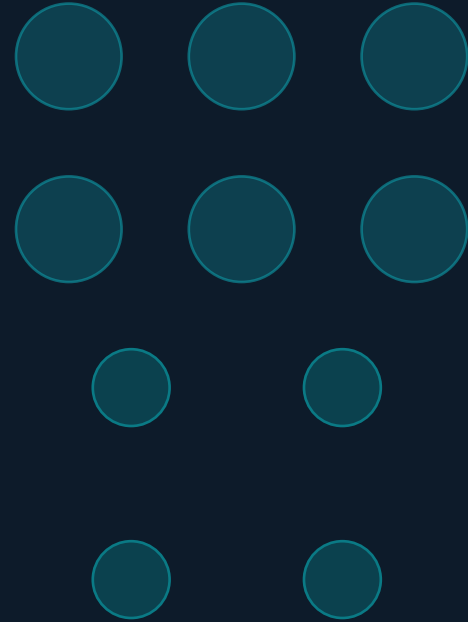


Adaptive Recommendation Engine

Coevolutionary Algorithms for Matrix Factorization

Two sub-populations (Users & Items) co-evolve to learn latent factor recommendations from the MovieLens ML-100K rating dataset



Problem Statement



The Challenge

How can a system automatically learn what users like from millions of ratings — without manual feature engineering?



The Goal

Build a recommender system that discovers hidden patterns in user-item interactions and predicts which items users will enjoy.



Our Approach

Use evolutionary algorithms where two populations (users & items) co-evolve their latent factor representations together.

What is Matrix Factorization?

The core mathematical idea behind the recommender system

The Rating Matrix

We have 943 users and 1,682 movies. Most ratings are unknown — we only have 100,000 out of ~1.6 million possible ratings.

Latent Factors

We decompose the rating matrix into two smaller matrices:

U (Users × 8 factors) — what each user likes

V (Items × 8 factors) — what each item offers

Predicted Rating = $U[\text{user}] \cdot V[\text{item}] + 3$

Movies →

?	4	?	5	?
3	?	?	?	4
?	?	5	?	?
?	3	?	4	?
5	?	?	?	4

↑
User

Rating Matrix (? = unknown)

How Coevolution Works

Two populations evolve together — each needs the other to measure its fitness

1

Initialize

Create 20 random
User matrices (943×8)
& Item matrices
(1682×8)

2

**Evaluate
Fitness**

Pair best User with
best Item $\rightarrow$ compute
RMSE on ratings

3

**Select
Parents**

Tournament or
roulette selection
based on fitness

4

Reproduce

Crossover + Mutation
 $\rightarrow$ produce new
child individuals

5

Survive

Elitism: keep best
of parents + children
 $\rightarrow$ next generation

$\cup$ Repeat for N generations (default: 50–100) $\rightarrow$ return best User & Item matrices

Evolutionary Operators

The building blocks that drive evolution in each generation

1 Selection

Tournament — pick best of k random candidates
Roulette — probability proportional to fitness

2 Crossover

Uniform — swap genes randomly
BLX-alpha — blend beyond parent range
DE — directional mutant vector

3 Mutation

Gaussian — add normal noise
Polynomial — SBM-style perturbation

4 Survival

Elitism — best of parents + children
Mu-Lambda — offspring only

5 Diversity

Fitness Sharing — penalise crowded
Crowding Distance — reward isolated

6 Representation

Real-valued — float latent factors
Binary — sign of latent factors (+1/-1)

Dataset & Configuration

943

Users

1,682

Movies

100K

Ratings

8

Latent Dims

ML-100K Dataset (MovieLens)

Parameter	Default
Population Size	20
Generations	50–100
Crossover Prob	0.8
Mutation Rate	0.15
Mutation Sigma	0.3
Regularisation λ	0.01
Bounds	[-3, 3]

Coevolution Modes & Diversity

Cooperative Mode

Users and items work TOGETHER.
Both populations try to minimise RMSE.
Item fitness = same direction as user fitness.
Best for finding globally optimal recommendations.

Competitive Mode

Items OPPOSE users — adversarial prediction error maximisation.
Item fitness = negated RMSE component.
Forces user population to become more robust.

Diversity Preservation

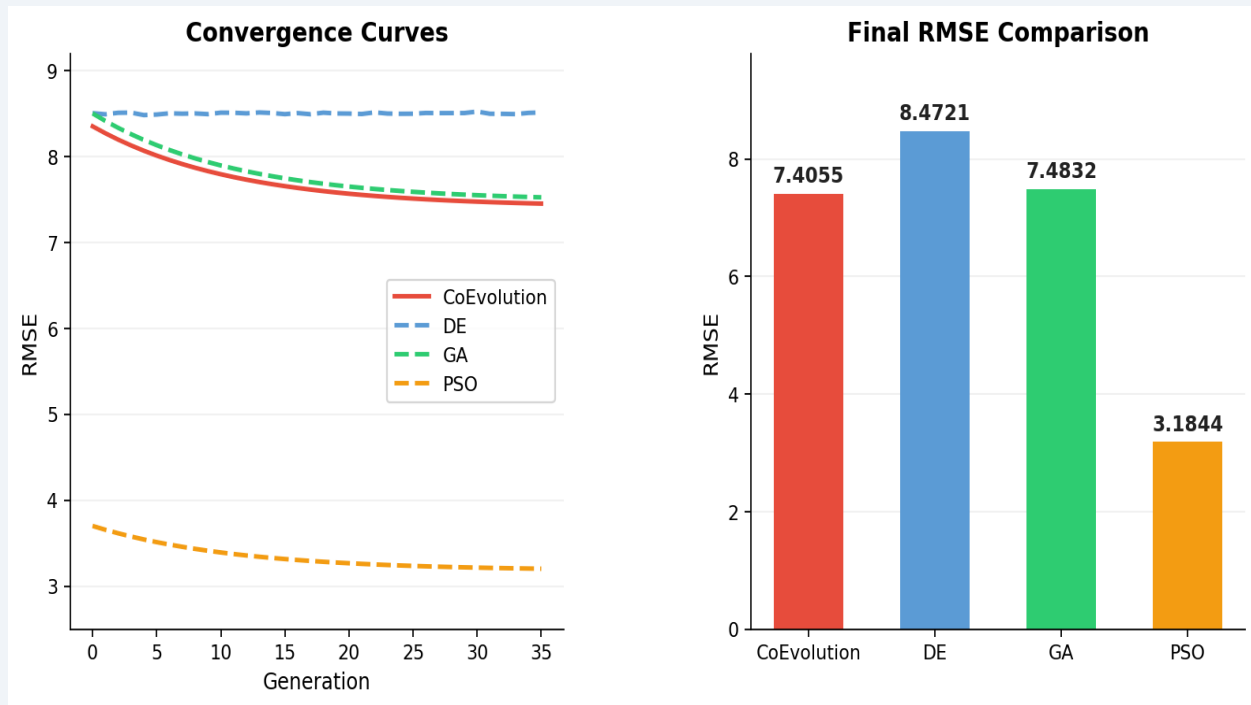
Fitness Sharing

Penalises individuals with too many similar neighbours — spreads the population out

Crowding Distance

Rewards isolated individuals on the fitness frontier — preserves boundary solutions

Comparative Analysis: CoEvolution vs Baselines



Key Findings

#1 PSO achieved best RMSE: 3.18

CE CoEvolution: 7.41 RMSE

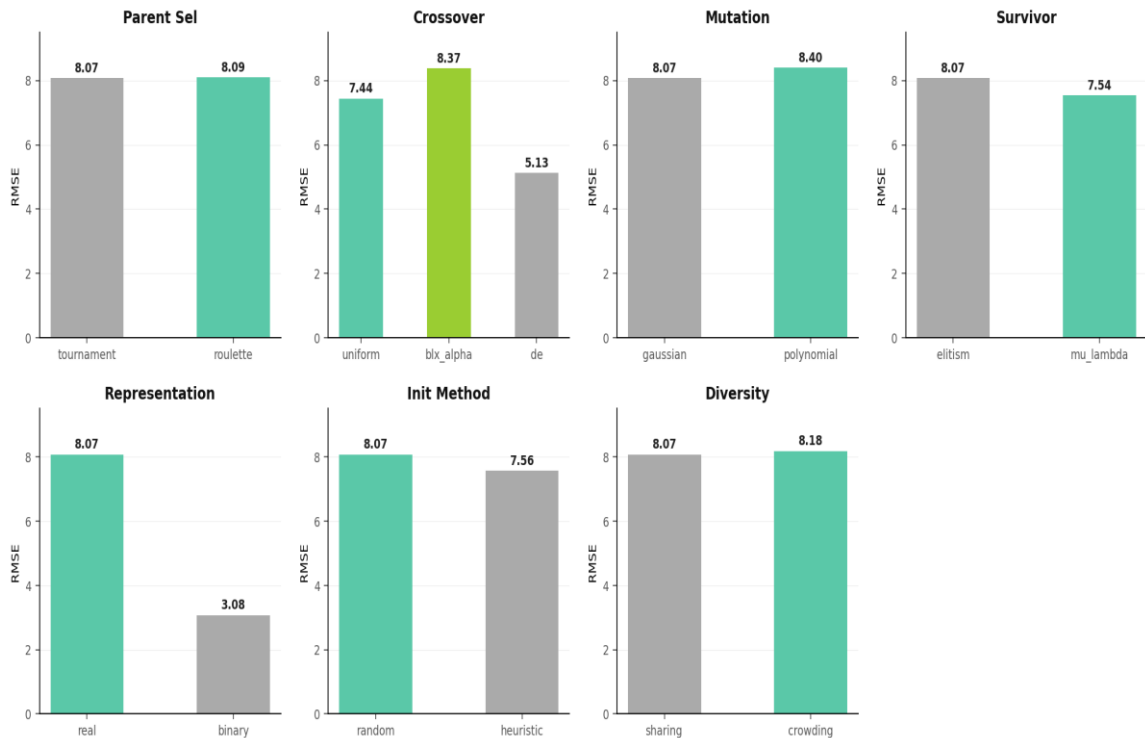
GA GA comparable to CoEvo: 7.48

DE DE performed worst at 8.47

Note: PSO excels at local refinement on this task. CoEvolution's value lies in its structured exploration of the latent factor space.

Automated Benchmarking Results

7 operator categories × multiple variants × 5 seeds each — RMSE mean ± std



Operator Insights

Crossover

DE crossover dramatically outperforms uniform & BLX- α

Representation

Binary encoding (± 1) yields much lower RMSE than real

Init Method

Heuristic init gives a significant head start

Parent Sel.

Tournament & roulette perform similarly

Survivor

Mu-lambda shows slightly better RMSE than elitism

System Architecture

Built as an interactive Streamlit web application

1 Run Evolution

Configure & run the coevolutionary algorithm. View live RMSE convergence, fitness curves, diversity plots, and top-5 recommendations per user.

2 Benchmarking

Automatically test all operator combinations across 30 seeds. Produces bar charts of mean $\pm$ std RMSE per setting.

3 Educational Mode

Side-by-side comparison of two different configurations — visualise how operator choices affect convergence and diversity.

4 Comparative Analysis

Run CoEvolution against DE, GA, and PSO baselines on the same data. Compare convergence curves and final RMSE.

Key Bug Fixes & Contributions

10 documented algorithmic fixes that correct the original implementation

1

Representative Selection

Best individual chosen by actual fitness, not hardcoded index-0

2

Competitive Fitness Sign

Regularisation always penalises; only RMSE negated in competitive mode

3

Roulette with Negatives

Shift-then-invert prevents equal-probability selection in competitive mode

4

Over-selection Bounds

replace=True prevents crash when bottom group is smaller than sample size

5

DE Population Size

Fallback to uniform crossover when pop < 3; random mandatory dimension

6

Polynomial Mutation

Correct SBM delta formula with proper sign on $u < 0.5$ branch

7

Sharing with Negatives

Sign preserved separately so crowded penalty works in competitive mode

8

Survivor Sort Direction

Absolute value used for competitive item fitness sorting

9

PSO Velocity Clamp

$v_max = 2.0$ prevents velocity explosion over many generations

10

Crowding Penalty Sign

Subtract (not add) crowding bonus to reward isolated individuals

Conclusion & Future Work

OK

Successfully built a coevolutionary recommender system with 10 operator choices and 2 coevolution modes

OK

Identified and fixed 10 algorithmic bugs in the original implementation

OK

Benchmarked all operator combinations — DE crossover and binary representation stand out as top performers

OK

Hybrid PSO integration provides additional local refinement on top of evolutionary search

Future Work

Multi-objective RMSE + diversity

Deep learning hybrid (neural MF)

Larger datasets (ML-1M)

Real-time adaptive mutation